开篇词 | 一个人用 Claude Code 造一个简版 Dify，需要多久？

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

先回答标题的问题：大概一周左右。

一个人，用 Claude Code，从零开始，搭建一个支持多模型管理、Agent 配置、对话交互、MCP 工具接入的 AI Agent 开发平台，我们叫它 Hify。基于 Spring Boot + Vue 技术栈，采用模块化单体架构，前后端完整，本地可运行。

你可能会想，Dify 是一个几十人团队做了两年多的产品，即便是“简版”，要覆盖核心功能链路，保守估算怎么也得一两个月、三五个人。一周一个人，听起来像是吹牛。

十分理解，如果半年前有人跟我说同样的话，我也这么想。但这门课不是来说服你相信的，而是把过程摊开给你看——怎么做到的，哪些地方 AI 真的帮了大忙，哪些地方它帮倒忙，最后由你自己判断。

角色转变：从“写代码”到“做决策”

我的本职工作是在一线大厂做基础设施，行业经验 10 年 +。过去几年，我一直在做一个开源的分布式系统项目。所以对于“一个人长期推进一个复杂系统”这件事，我并不陌生。以前纯靠手写，即便每天下班后写两三个小时，推进速度也很慢。

Claude Code 出来之后，我最初的用法和大多数人一样——把它当一个更聪明的代码补全工具。问它函数怎么写，让它帮我重构一段逻辑，或者生成一些样板代码。确实有提效，但说实话，提的有限。因为核心瓶颈不在写代码的速度上。

真正的转变发生在我开始尝试用它做一个完整项目的时候。

我想做一个简版 Dify，一方面是因为我需要一个可以本地部署的 Agent 平台来对接工作中的场景，另一方面也是想验证一件事：Claude Code 到底能不能扛住一个有真实复杂度的系统工程?

第一天，我就踩了一个大坑。

我直接告诉 Claude Code：“帮我设计一个 AI Agent 开发平台的后端架构，支持多模型、多 Agent、工具调用。”它很快给了我一个方案：微服务划分清晰，技术选型合理，甚至画了架构图。看起来很专业。

但我越看越觉得不对：它给的方案太重了，上来就引入了服务注册中心、分布式配置、链路追踪，好像在做一个要服务百万用户的生产系统。而我只需要一个团队内部几十人用的平台，模块化单体就够了。

问题不在 Claude Code 身上，问题在我没有想清楚就让它开始干活。

1. 我没有定义清楚产品边界：什么做、什么不做；
2. 没有明确架构约束：这是一个一周要交付的单人项目，不是一个要演进三年的团队项目；
3. 没有给它任何规范：命名怎么定、接口怎么设计、错误怎么处理。

它不是不能做好，是我没告诉它“好”的标准是什么。

那一天我几乎没写一行代码。我花了整个晚上做三件事：定义产品边界（一页纸写清楚做什么和不做什么）、设计核心数据模型（五六张表的 ER 图）、写 CLAUDE.md（告诉 Claude Code 这个项目的全部上下文和规范）。

第二天，一切都不一样了。

我给 Claude Code 下达的不再是模糊的大需求，而是清晰的小任务：“按照 CLAUDE.md 中的接口规范，实现模型提供商的 CRUD 接口，包含连通性测试功能，异常按错误码区分三种情况。”它的输出质量发生了质的飞跃，不是因为它变聪明了，而是因为我给它的输入变清晰了。

到这里我才真正体会到：**用 Claude Code 做项目，瓶颈从来不在 AI 的能力上，而在你的思考质量上。**你想得越清楚，它做得越准确；你越模糊，它越跑偏。

这不就是架构师和工程团队的关系吗？Claude Code 就是你的工程团队，而你必须成为那个架构师——决定做什么、不做什么、用什么方式做、达到什么标准。

方法论：不只是写代码，而是拆解问题

成为架构师之后，你每天做的最多的事不是写代码，而是拆解问题：**这个系统的边界在哪、这个需求分几步、每步交给 Claude Code 的输入是什么。拆得好，它一次做对；拆得差，你反复返工**。

这门课教的就是怎么拆，怎么和 Claude Code 一起探讨、一起拆，然后得到一个好的功能结果，而不是代码结果。

如果你期待的是一门 Claude Code 功能介绍课——教你怎么用 Hooks、怎么配 Skills、怎么写 Prompt——那这门课可能不是十分匹配你的需求。那些内容我们会讲，但不是重点。这门课教的是一套方法论——**当你拿到一个复杂系统的需求，如何把它拆解成 Claude Code 能准确执行的任务链，一个人从零交付出来**。我管它叫“Claude Code 全链路开发框架”。

这套框架包含两种协作模式：

- 执行模式是你拆清楚了让它做，你验收；
- 咨询模式是你还没拆清楚，先让它帮你梳理“这一步应该考虑什么”，你判断取舍后再让它执行。

框架还包含一系列支撑拆解过程的具体方法论：

- 规范驱动开发（SDD）让 AI 永远在你定义的轨道上，不自由发挥；
- 三问裁剪法帮你拆解产品边界——做什么、不做什么、做到什么程度；
- 三步检查法帮你高效审查 AI 输出——先查意图、再查质量、最后查边界。

这些方法论不只适用于 Hify，换成任何项目都能用。

我会带你经历 Hify 从无到有的全过程。但在每个环节，我关注的不是“这个功能怎么实现”，而是问题是怎么被拆解的：

- 在动手之前，我是怎么让 Claude Code 帮我梳理 Dify 的功能全景、怎么用三问裁剪法确定 Hify 的边界、怎么让它给架构方案然后我来拍板的?
- 在核心开发阶段，我是怎么把一个大需求拆成 Claude Code 能准确执行的小任务的？拆的粒度怎么定？它跑偏了我怎么拉回来?
- 在基础设施阶段，我怎么用咨询模式让 Claude Code 帮我发现遗漏的组件?
- 在联调和测试阶段，当问题出在模块之间的交互而不是单个模块内部时，我怎么给它足够的上下文让它定位问题?

每节课以图文为主体，讲清楚方法论和思考过程。每个大章节结束后，配一个完整的演示视频，展示真实操作过程。图文让你理解方法，视频让你看到现场。

> 提示：课程以 Claude Code 为核心教学工具。建议优先使用 Claude Code 跟随实践，学习体验更佳。如果没有开通，也可选用其他 AI 编程工具——方法论是通用的，实现效果有

> 差异而已。

用 Claude Code 造一个简版 Dify

Hify 项目 · 完整课程结构

第一部分：认知篇

你是架构师，AI 是执行团队

- 三层分工模型
- 规范驱动开发 SDD
- CLAUDE.md 机制

第二部分：顶层设计篇

动代码之前，想清楚做什么、怎么做

- 产品定义
- 架构设计
- 规范体系
- 实操演示

第三部分：工程搭建篇

从空目录到基础设施就绪

- 后端骨架
- 前端工程
- 基础组件
- 熔断线程池
- 实操演示

第四部分：核心功能交付·基础篇

从第一个功能到智能客服开口说话

- Provider 管理
- 领域理解四问
- 模型适配层
- Agent 配置
- 对话引擎 SSE
- 上下文管理
- 前端管理页
- 对话页面

第五部分：核心功能交付·高阶篇

RAG、工作流、MCP — 企业级能力

- RAG 知识库
- 工作流编排引擎
- MCP 工具接入
- pgvector + Embedding
- JSON → 执行引擎
- Function Calling

第六部分：测试与部署

从本地开发到生产可交付

- AI 写测试找 bug
- 容器化 + K8s 部署
- 可观测性监控

第七部分：高阶与复盘



AI 是放大器，不是发动机

最后，在课程正式开始之前，我想告诉你一件事。

构建 Hify 的过程中，Claude Code 确实帮我做了大量的事。但也有不少地方它帮不上忙，甚至帮倒忙，比如架构选型给过明显过度设计的方案，跨模块的状态一致性问题反复改了几轮都没改对……

我自己的实践经历让我确认了一件事：**AI 是放大器，不是发动机**。你有方向它放大十倍，你没方向它放大的是零。

能一周交付 Hify，不是因为 Claude Code 厉害，是因为动手之前就知道这个系统应该长什么样。哪些模块必须有，哪些功能可以砍，数据模型怎么设计——这些判断来自自身的经验积累，Claude Code 可以帮我们把判断高效地变成代码，但判断本身是我们自己的。

所以进一步来看，你能收获的，不仅是怎么用好 Claude Code，更重要的是，怎么做好那个下判断的人。有了这个能力，你不会绑定任何一个工具，未来换了更好的 AI 编程工具，经验依然可复用。

VibeCoding 时代，浪潮奔涌向前，工具迭代飞快。希望这门课程能助你夯实核心竞争力，享受 AI 带来的乐趣！

理性看待 Claude Code 源码泄漏事件

课程上线前一天，Claude Code 出了个不大不小的新闻：v2.1.88 版本发布时，npm 包里意外带上了一个 60MB 的 source map 文件，把大约 51 万行 TypeScript 源码、1900 多个文件直接暴露了出来。GitHub 上的镜像仓库几小时内被 fork 数万次，各路媒体、安全研究者、吃瓜群众纷纷下场，热度拉满。

原因很简单：Bun 打包器默认生成 source map，发布时没在 `.npmignore` 里把它排除掉。一行配置的问题。Anthropic 官方确认是“人为错误导致的打包问题，不是安全漏洞，没有用户数据或密钥泄露”。

作为一个正在做 Claude Code 课程的人，我觉得有必要聊几句真话，免得有人被热点带偏了节奏。

泄漏的是客户端，不是大脑。

这次暴露的是什么？是 Agent 的客户端工程实现——工具调用逻辑、权限系统、上下文拼接策略、多 Agent 协作架构。社区也挖出了一些藏在 feature flag 后面的未发布特性，比如后台常驻的 KAIROS、远程规划的 ULTRAPLAN 等等。

有意思吗？有意思。但这些东西改变了什么吗？什么都没改变。

Claude Code 之所以强，是因为背后的 Claude 模型在代码理解和生成上的能力。这个能力来自模型训练、海量数据和算力投入，来自 Anthropic 在 AI 安全与对齐上的深层积累。这些东西不在源码里。竞争对手看到了客户端架构，能学到 Agent 编排的工程思路，但学不到“为什么 Claude 比其他模型更擅长写代码”。工程可以复制，模型能力不能。

至于那些未发布特性，本来就在产品路线图上，早晚会公开。提前知道了，也就是看了个预告片。而且以 Claude Code 几乎日更的迭代速度，这份源码一周后可能就跟最新版对不上了。

这恰恰印证了我们课程的核心判断。

这件事很好地说明了一个道理：工具层面的东西，无论是客户端架构还是具体的功能实现，都是会变的、可替代的。51 万行源码摊开在所有人面前，Claude Code 的竞争力一点没少。因为真正的壁垒不在工具怎么实现，而在模型能力，以及使用工具的人怎么思考。

这也是我做这门课的出发点。我们不是在教一个工具的使用方法。如果是，那源码泄漏确实会让课程贬值，大家自己看源码就行了。但我们教的是方法论：怎么拆解问题、怎么定义边界、怎么用规范驱动 AI 在你定义的轨道上跑，怎么做那个下判断的人。

这些能力不会因为某个工具的源码泄漏了就贬值，也不会因为工具出了新版本就过时。恰恰相反，工具迭代得越快，掌握底层方法的人越值钱，因为只有他们能在工具更替中始终保持生产力。

所以，热闹归热闹。看完之后，我们来聊点真正重要的事：怎么在 Vibe Coding 时代，成为那个不被工具绑定的人。

现在课程正式开始。

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (38)

最新　精选

edward
2026-04-01 来自湖南

很期待老师这样的课程，希望老师后面还能讲下存量遗留项目如何使用这套方法论做迭代开发。

作者回复: 好主意。我看看再最后一个模块专门讲一下这个。这个特别值得讲。如何了解当前项目，

如何review pr，很有意思的议题

共 4 条评论>

17



菲尔德

2026-04-15 来自贵州

最近遇到个生产问题，用python开发的Agent工程内存泄露，生产环境不让做变更，用了不少工具和方法，2天都没定位到问题点。跟同事讨论起都说，当时选型用java开发就好了，Arthas非侵入式，太好用了。
复杂业务+长稳要求，python可能真不如Java。
还有一点就是python太灵活了，写上来的代码非常个人定制化，当时着急上线，现在维护起来，也是一堆问题。

作者回复: 个人认为，业务系统 java 永远的神，不接受反驳～ hhhhh

6



Tomorrow

2026-04-11 来自上海

哈哈哈好喜欢老师的文笔和叙事节奏，请问这也是Claude Code写的吗（bushi）

作者回复: 老实说，这个还真不是，哈哈哈，Claude Code写的你们应该能看出来。AI写的东西可以看出AI味道的。很直观的。

6



humor

2026-04-29

我花了整个晚上做三件事：定义产品边界（一页纸写清楚做什么和不做什么）、设计核心数据模型（五六张表的 ER 图）、写 CLAUDE.md（告诉 Claude Code 这个项目的全部上下文和规范）

老师说的整个晚上是指通宵达旦一晚上没睡觉吗？还是晚上的两三个小时啊

作者回复: 两三个小时，哈哈哈哈

3

Geek_082133
2026-04-13 来自浙江

老师，everything claude code，superpowers，oh my claude code这三个等插件技能是不是也几讲下，或者有比较好的使用文档

作者回复: 我记录下了，我后面加一下，挺好的建议。感谢～

3

silence
2026-04-02 来自广东

想解决的问题
– 怎么更好地发挥 claude code 的能力，一个人交付整个系统

想获得的启发
– 借鉴作者的思考过程，借助 AI 工具，构建一个自己的产品
– 学习作者的方法论，内化并使用

对课程介绍的感受
– 很期待这门课，课程的目标很让人兴奋
– 也有一点想为 Claude Code“发声”
  – Claude Code 不重要，重要的是后面的模型，从一个角度不是太同意
  – 有一个观点，Agent = Model + Harness，Claude Code 就是 Harness，使用同样的模型，效果不同的原因就在于 Harness 不同
  – 大模型的发展，后续很可能差距会逐渐缩小。同样的模型在不同人手里产生的效果不同。使用不同的 harness 工具产生的效果会不同。所以 Claude Code 其实也蛮重要的。
  – 个人使用 Cursor 时也用的 Claude 的模型，但是实际效果没有 Claude Code 的好
– 非常期待作者的课程内容，希望借鉴作者的思考过程，借助 AI 工具，构建一个自己的产品，学习作者的方法论，内化并使用

作者回复: 哈哈哈，感谢期待～。
“Claude Code 不重要，重要的是后面的模型”。讲这句话的时候我想表达的是：我希望这门课传递给大家的是思考的过程，Claude Code 只是我们的工具，我们用这个工具，用我们的思考的过程，我们的方法论去达到目的。

我非常认可你说的，大模型的发展，后续很可能差距会逐渐缩小。因此才会说这句话。我希望的是大家有了这些思考的逻辑和思维后，即使不用claude code也能达到我们的目的。

我主要是想强调，我写这门课的时候，希望大家可以在这个过程一直在思考，然后学到经验。重点不是claude code，是思考的过程。

2



鲲鹏飞九万里

2026-07-12 来自日本

这篇文章中，怎么一会儿 Dify ， 一会儿 Hify

作者回复: hify是一个简化版的dify，也就是这门课要做出来的作品～

1



文艺理科生

2026-05-25 来自美国

老师，看了几节课，发现使用AI工具的差距还在于技术决策，技术决策依赖于技术敏感度，即根据一个业务场景是否能够快速想到有哪些核心问题需要考虑。而技术敏感度的培养，当下的资深架构师得益于前些年没有AI工具时代可以慢慢磨，但现在有了AI工具，项目不会给未成为资深架构师的人这种过程再去成长，而是提效、速成，而技术敏感度是需要时间、经历的项目、不断思考慢慢沉淀的。所以请问在AI时代下，如何还能在传统专家培养模式失效的前提下，还能继续培养技术敏感力?

作者回复: 看到你问我答案，我挺开心的，因为我觉得你在深度思考这个问题。超级超级超级超级超级好的问题。这也是我一直一直在想的一个问题。

我就说一下我的个人看法（个人局限～～）。在这个问题的基础上，我有两个问题：
1. 对于“当下的资深架构师”即使有前面的积累，慢慢磨养成了技术敏感度。但是在未来AI用多了，技术敏感度下降后，怎么办?
2. 对于“还没成为资深架构师”的同学怎么在AI时代追求速度的场景下，培养自己的技术敏感度呢，他们怎么办呢?

再衍生一个问题：当所有人都会使用AI工具，熟练AI编程后，我们这些程序员的核心竞争力是什么?

我今天给我自己的答案是：
1. 从个人角度： 卷呗，兄弟们。我一直就认为程序员就是体力活，多学、多练、多花时间，多做。AI编程也得比熟练度是吧，AI 编程也得解决深度问题吧，就是你说的技术敏感度，技术决策。卷、多花时间培养自己的这些东西。
2. 从落地的角度：就是多做项目。多花时间去做项目。多花时间做公司的项目，做开源项目。公司没

合适的，就搞开源，只要你花的时间够多，那你的这个问题就解决了。

所以答案就一个把：卷。怎么卷，卷公司代码，卷社区代码，多写多练。哈哈哈哈，个人意见～

共 2 条评论 >

1

星辞Sting

2026-04-26 来自浙江

很期待老师的课程，作为开发工程师，也在2024年就开始做AI应用的人，我一直想怎么能更好的用AI。很多课程挂羊头卖狗肉都是讲怎么调用API、RAG、MCP等基础科普知识，也花过不少冤枉钱。 就是想找老师这样内容，如何拆解问题能让AI更好的工作。 期待后面学习能和开篇词里的内容一样

作者回复: 期待你看下去。我也期待你的反馈。我也是想做一个真正有价值的课。如果写的不好的，不符合预期的地方，请告诉我，我来改～。

1

尔东橙

2026-06-28 来自上海

会讲一下dify整体技术架构么，比如我都不完全清楚dify的各个层面的实现，我怎么知道cc 设计的好不好呢

作者回复: 课程里面会讲一下dify是怎么做的，但是不会转门对dify的代码进行讲解。

cc 设计的好不好，在我看来就得看我们自己的积累、经验和理解了。

对我来说，对不清楚的东西，我的做法是，和cc一起做个技术调研，然后结合自己的经验，我觉得基本就能判断设计的好不好。